

CSOPESY -- 1 Introduction to Operating Systems

What is an operating system?

- Program, intermediary between user and computer hardware

Goals:

1. Execute user programs and make solving problems easier
2. Make computer system convenient
3. Use computer hardware in an efficient manner
 1. Manage power consumption or battery life
 2. Try to manage throttling of CPU, for example, Parking the cores if di kailangan. Bale parang free loaders sila pero ok lang for saving power and conserving energy.
Trabaho iyan ng operating system

Computer Structure

- Hardware
 - CPU, Memory, I/O, Storage
 - Essentially, lahat ng na sa task manager mo
- Operating System
 - Controls and coordinates use of hardware
- Application Programs
 - define ways which system resources are used to solve the problem
 - word processors, browser, compilers, video games
- Users
 - Literally the ones using the OS
 - OS allows multi-user environment

Computer Structure

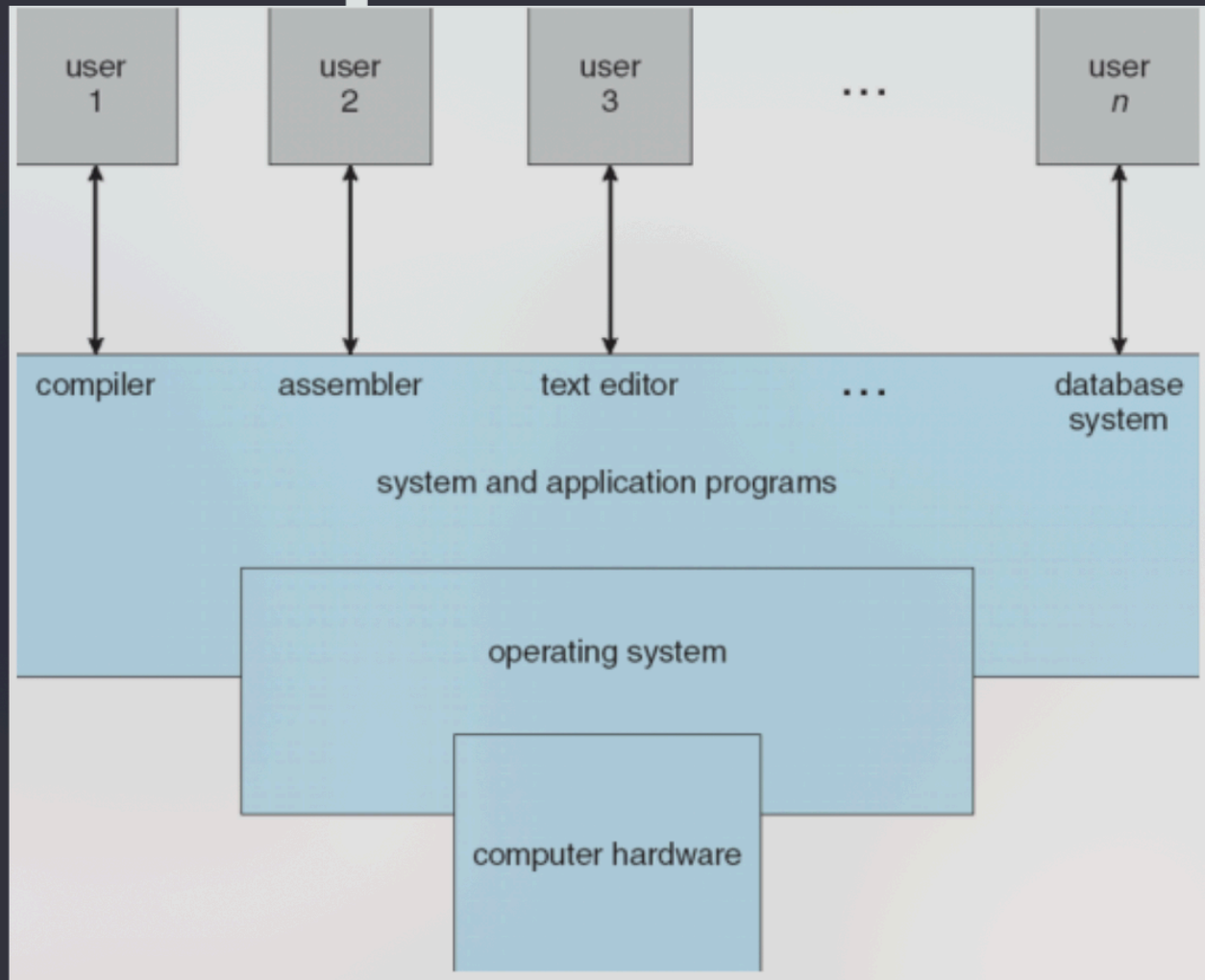


Image 1: Computer Structure with users

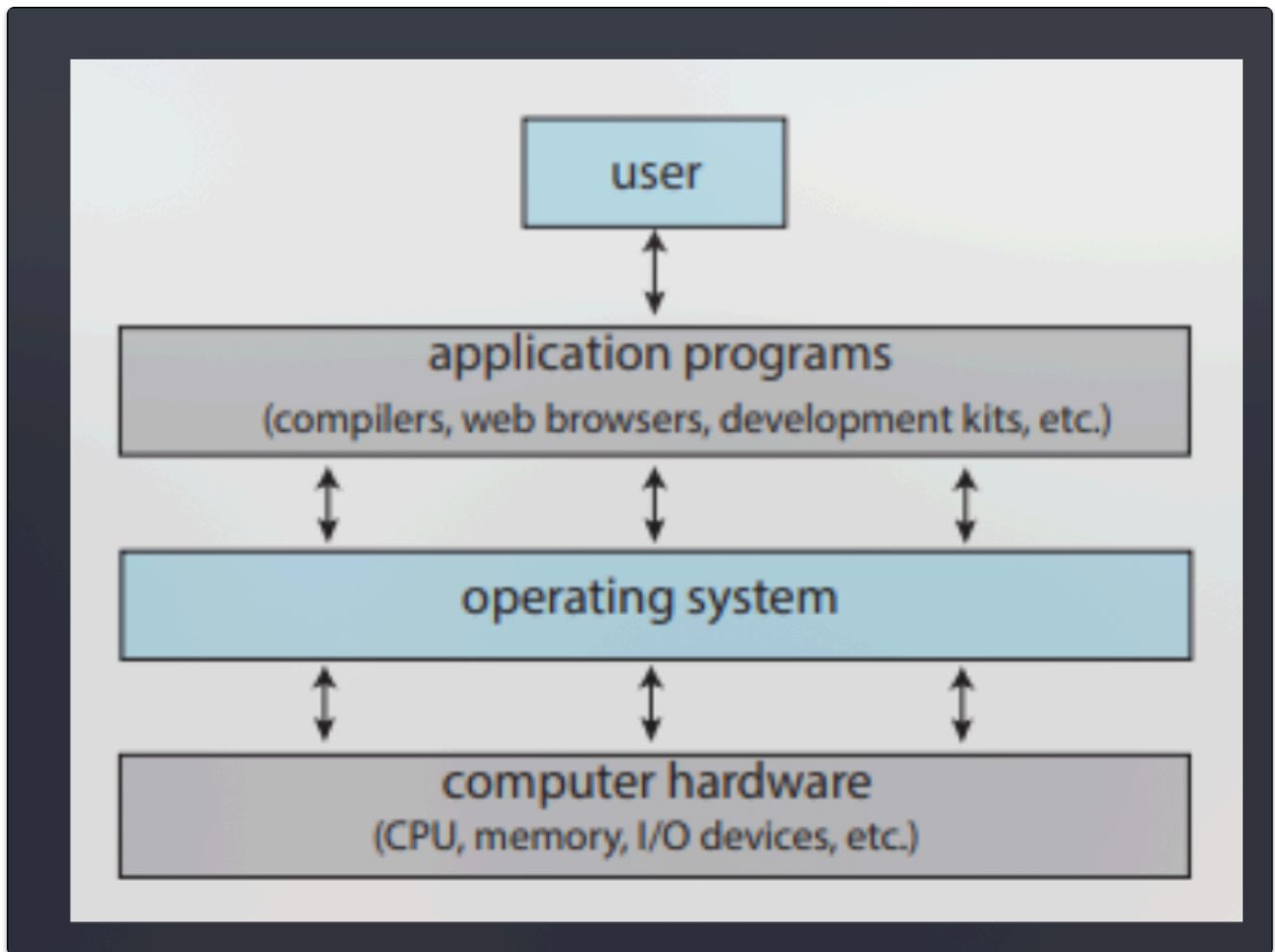


Image 2: How hardware and applications communicate using OS

Why Study OS?

- all code runs on top of an operating system
- studying OS gains additional knowledge that can lead to proper, efficient, effective and secure programming

Operating System Definition

1 Operating System is a resource allocator

- Siya ang nagb-budget ng resources
 - Think of your weekly allowance
- Decides between conflicting requests for efficient and fair use
- Manage resources such that some process can run

2 Operating System is a Control program

- Controls execution of programs to prevent errors
- Worst case scenario, BOSD if OS is being improperly used
-

3 OS is the one program running at all times on the computer (i.e., the kernel)

- kernel is core of any modern operating system
 - cpu scheduling, memory management
- System programs - software not part of kernel but associated with OS
 - Windows Explorer is part of OS. If dinelete siya sa taskbar, mawawala yung UI. Its not a critical part of the OS but not really needed (i.e., server only distribution)
 - Task Manager, built into the OS, but not really necessary to running the OS.
- Middleware - set of software frameworks that provide additional devices
 - ALUs, Carry Look-ahead
- Application programs
 - Anything not necessary for the usage of the operating system

Process Management

- Create, delete user processes (i.e., Offing explorer)
- Scheduling processes and threads in CPUs

4proc	filter	reverse	tree	< cpu lazy >
Pid:	Program:	User:	MemB	Cpu% ↑
42290	python3	moonberry	46M	0.2
23110	Isolated We	moonberry	513M	0.1
22232	firefox	moonberry	745M	0.1
42158	kitty	moonberry	200M	0.1
22771	Isolated We	moonberry	596M	0.0
22407	WebExtensio	moonberry	156M	0.0
1957	obsidian	moonberry	375M	0.1
622	Hyprrland	moonberry	151M	0.3
37092	kworker/u32	root	0B	0.0
42163	kitten	moonberry	24M	0.0
20597	kworker/u32	root	0B	0.0
1929	obsidian	moonberry	126M	0.0
22623	kworker/u33	root	0B	0.0
24553	Web Content	moonberry	81M	0.0
23858	Web Content	moonberry	81M	0.0
23853	Web Content	moonberry	81M	0.0
23193	Isolated We	moonberry	118M	0.0
22329	Privileged	moonberry	115M	0.0
29888	kworker/u33	root	0B	0.0
734	waybar	moonberry	50M	0.1
1861	obsidian	moonberry	167M	0.0
751	Xwayland	moonberry	73M	0.1
447	irq/184-MSF	root	0B	0.0
22333	RDD Process	moonberry	60M	0.0
38382	kworker/0:1	root	0B	0.0
20609	kworker/6:3	root	0B	0.0
22451	Utility Pro	moonberry	58M	0.0
26268	kworker/0:0	root	0B	0.0
31308	kworker/3:2	root	0B	0.0 ↓
↑ select ↓	info ↓	Terminate	Kill	0/232

Image 3: Linux task manager using bpytop (I use arch btw)

- Suspending and resuming processes
 - Example, if naggenshin ka and naoff yung screen mo sa phone, youll get back to the splash screen
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication

Memory Management

- Keep track which parts of memory are currently being used
- allocating and deallocating memory space as needed
- deciding which memory needs to be into and out of memory

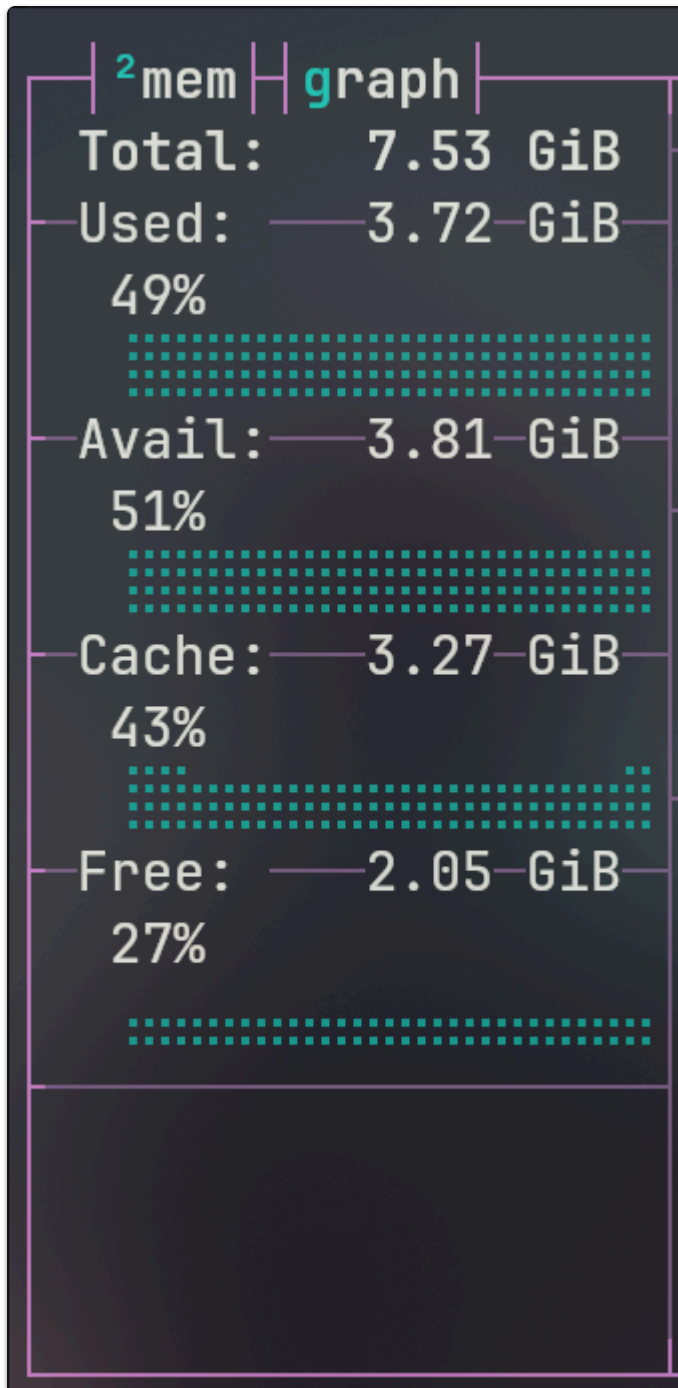


Image 4: Memory usage (in bpytop, using arch btw)

- General purpose computers run most programs from rewritable memory, called main memory (RAM)
 - Main memory - commonly implemented in a semiconductor tech called dynamic random-access memory (DRAM)
- We follow Von Neumann Architecture where the CPU cannot process something that is not in memory.

- fetch instruction from memory, store instruction in instruction register. instruction is then decoded and may cause operands to be fetched from memory and stored in some internal register
- After instruction of operands is executed, result may be stored back into memory

Storage Definitions and Notation Refresher

Basic unit is **bit** - 0 or 1

Byte is 8 bits

Word is one or more bytes

Kilobyte is 1024

Megabyte is 1024^2

Gigabyte is 1024^3

Terabyte is 1024^4

Ideally, we want the programs and data to reside in main memory permanently. This arrangement usually is not possible on most systems for two reasons:

1. Main memory is usually too small to store all needed programs and data permanently.
2. Main memory, as mentioned, is volatile—it loses its contents when power is turned off or otherwise lost.

Most computers provide secondary storage as extension of main memory. Most common are Hard Disk Drives - HDDs and Non Volatile Memory - NVM's devices

File System Management

- Creating and deleting files
- Creating and deleting directories to organize files
- Supporting primitives for manipulating files and directories
- Mapping files onto mass storage
- Backing up files on stable storage

Cache Management

- Temporary storage that is fast (i.e., registers, cache, memory disks)

- OS manages hierarchy of storage

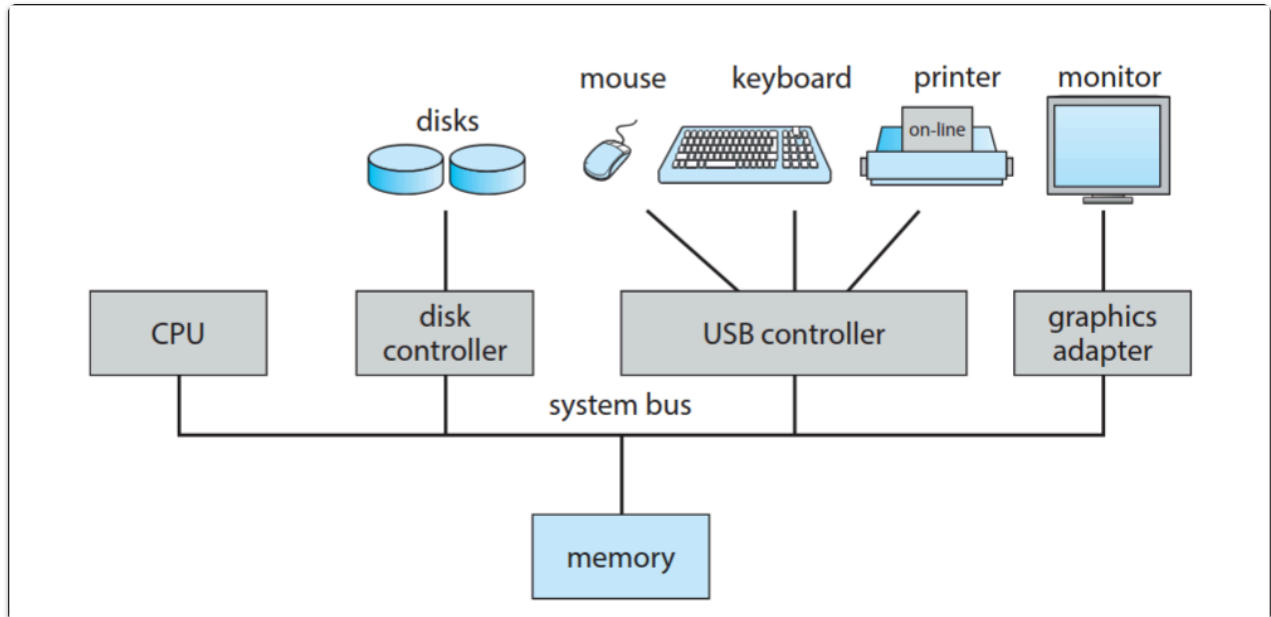
Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

- OS are actually responsible for NCQ (native command queueing), kung paano umiikot yung disk, si OS rin nagschedule niyun.
 - Minsan depende rin sa laki ng drive

IO System Management

- Input devices also handled by OS
- Component that includes buffering, caching and spooling
- Modern general purpose computer consists of device controllers connected through a common bus
 - access between components and shared memory
 - depending on controller, a lot of devices can be connected
 - system USB port can connect to USB hub, several devices can connect to that
 - maintains local buffer storage and set of special registers
- General device-driver interface
- Drivers for specific hardware devices
 - N-Key Rollover from gaming keyboards
 - Back then KBs cant handle simultaneous input na apat, pero what if youre an esports player with high APM? The kbs cant keep up
 - What they did was KBs got buffer circuits added to them. Keyboard manufacturers added custom buffers but now they had to install custom drivers.

- Drivers expands capabilities of the operating system



OS Services

- OS provides execution environment for programs by providing services available
- Available services available varies depending on operating system and hardware

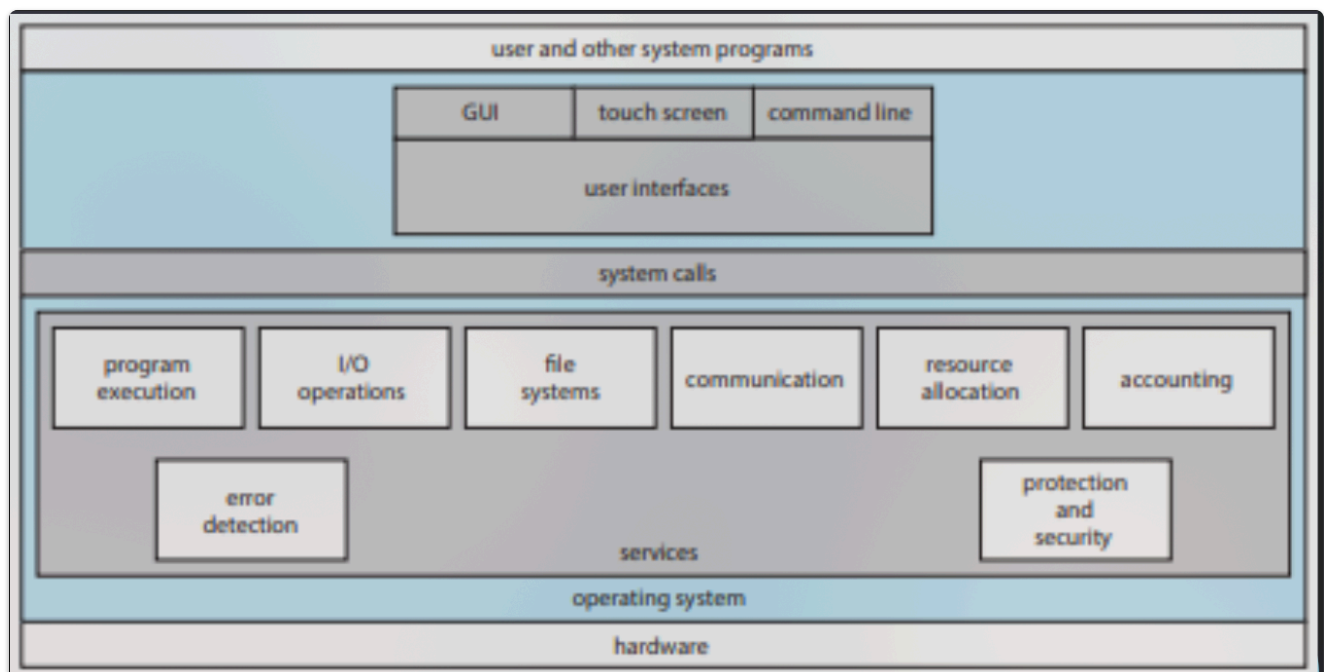


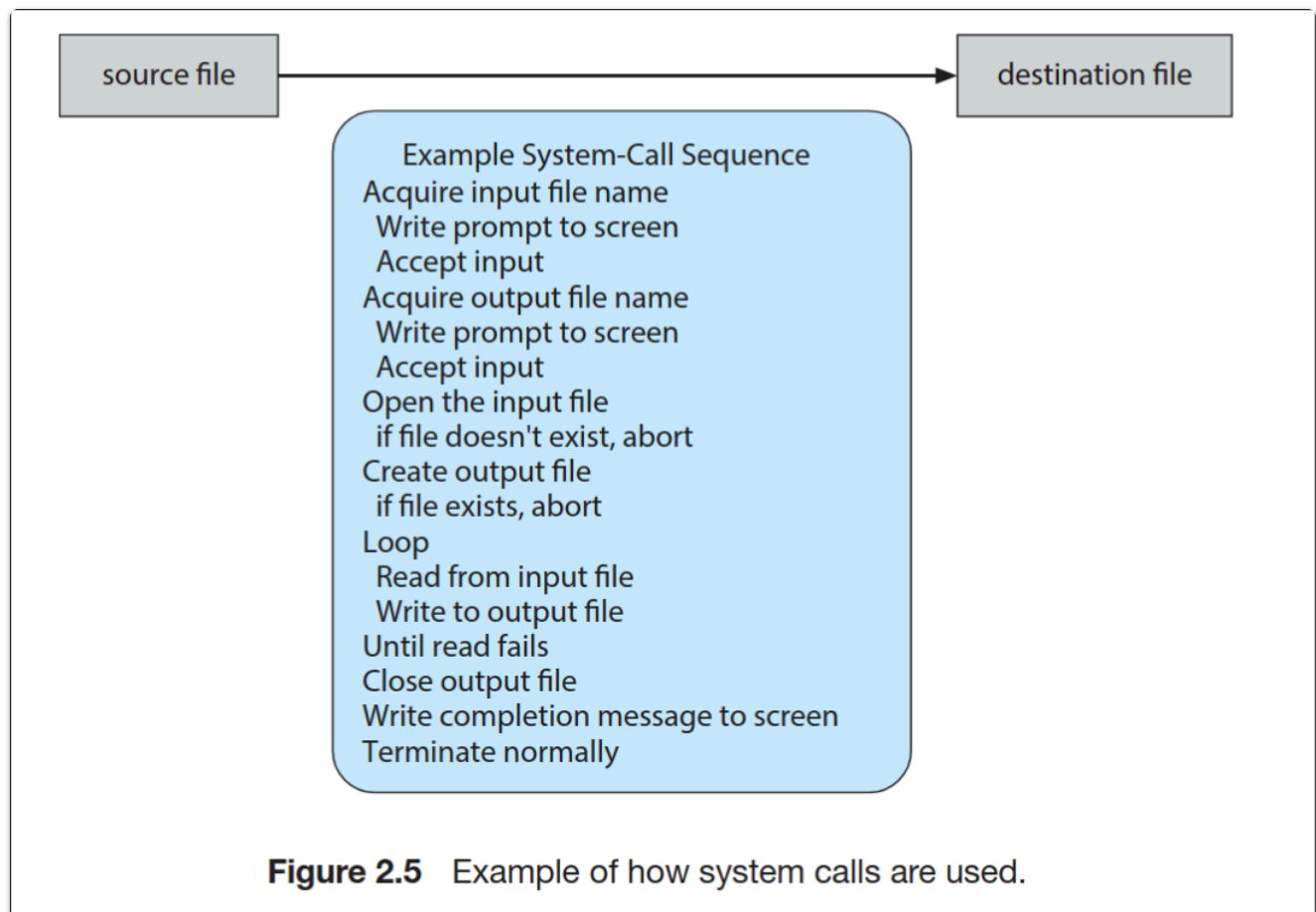
Image 5: All the systems calls are provided by the OS

System Calls

- API ng OS to the world
- Parang assembly programming language but higher level

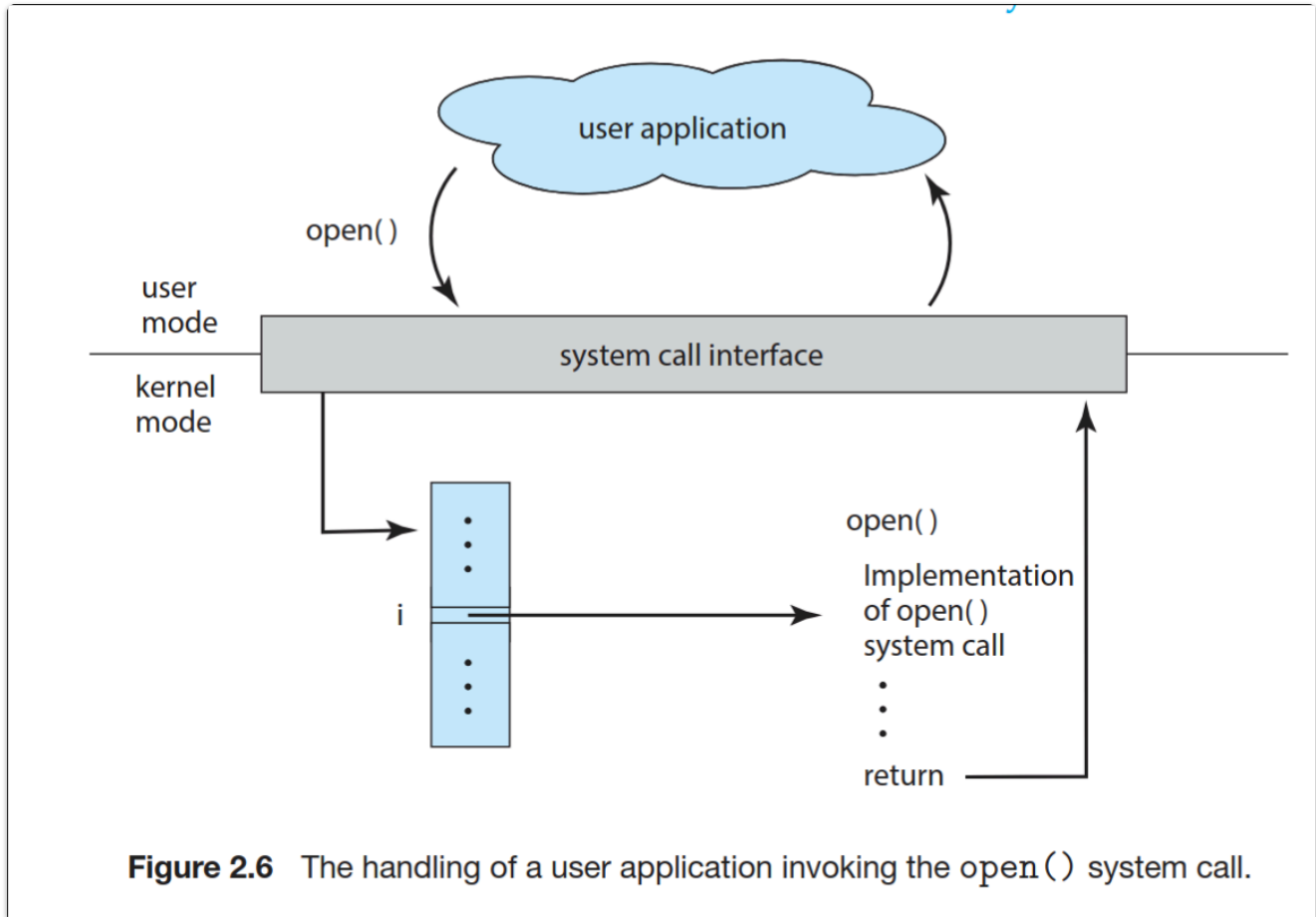
Kapag cinompile lahat ng system calls, you create a compiler

An example of a system call is doing the command `cp in.txt out.txt`
`cp` in linux is copying a file, so this copies `in.txt` content into `out.txt`
So opening the `in.txt` file is a system call, accessing `out.txt` file also a system call. Error handling is a system call



API - application programming interface - set of functions that are available to an application programmer, including the parameters that are passed to each function and the return values the programmer can expect

Here is how API calls are handled by the OS



System call interface serves as a link to system call made available to OS and Run Time Environment.

- This is why applications are not cross-compatible across OS'. Iba-iba kasi system calls ng OS

Portability

- Due to variances of system calls from one OS to another
- Portable applications are still possible using following techniques
 - Interpreted programming language (python, ruby, nodejs), interpreter reads each line of program and executes equivalent instructions native to OS
 - Yes may overhead and theoretically a bit slower pero computers are now faster
 - PL includes its own full Runtime Environment (Java) - RTE abstracts system calls from program
 - Standardized use of API related to compilation of binaries (POSIX, UNIX)

OS Structures - Monolithic

- All functionality is in a single static binary file
- I.e., Linux
- Tightly Coupled

- If something goes wrong in any component, then it will crash

OS Structures - Layered

- Modular components - layered where lowest layer is the hardware and highest layer is UI
- Loosely coupled
- Changes can be modular

I.E., Vanguard from Riot Games is said to be Kernel-layer anticheat

Kernel layer can see outward but no one can see inward

Another example, shutting down Windows Explorer. It can be closed without bringing the OS with it kasi its in the outer ring

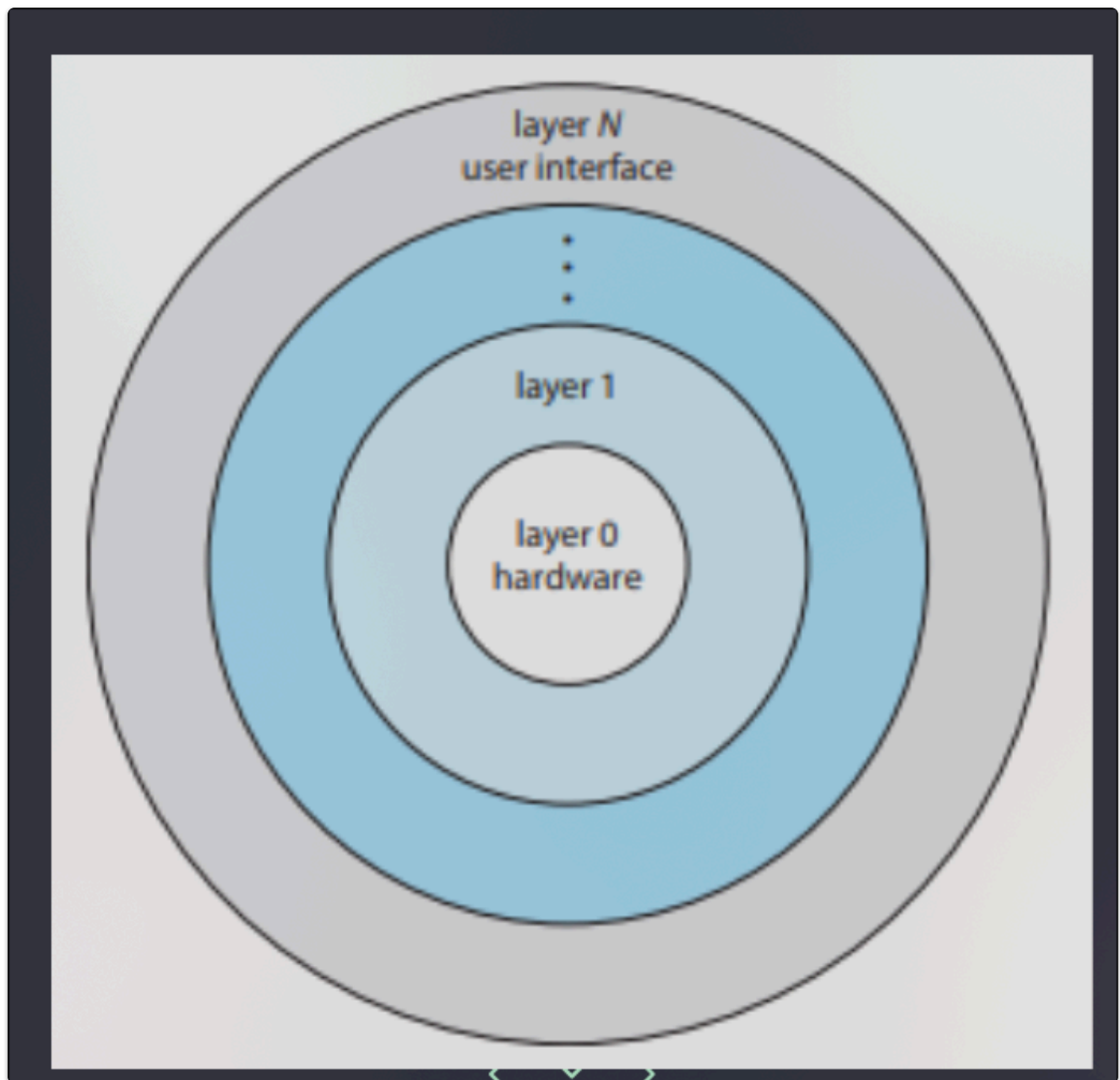


Image 6: Layering

Microkernels

- Carnegie Mellon University developed an OS called Mach that modularized kernel using micro-kernel approach
- OS resistant to failure

New services are added to the user space without modification

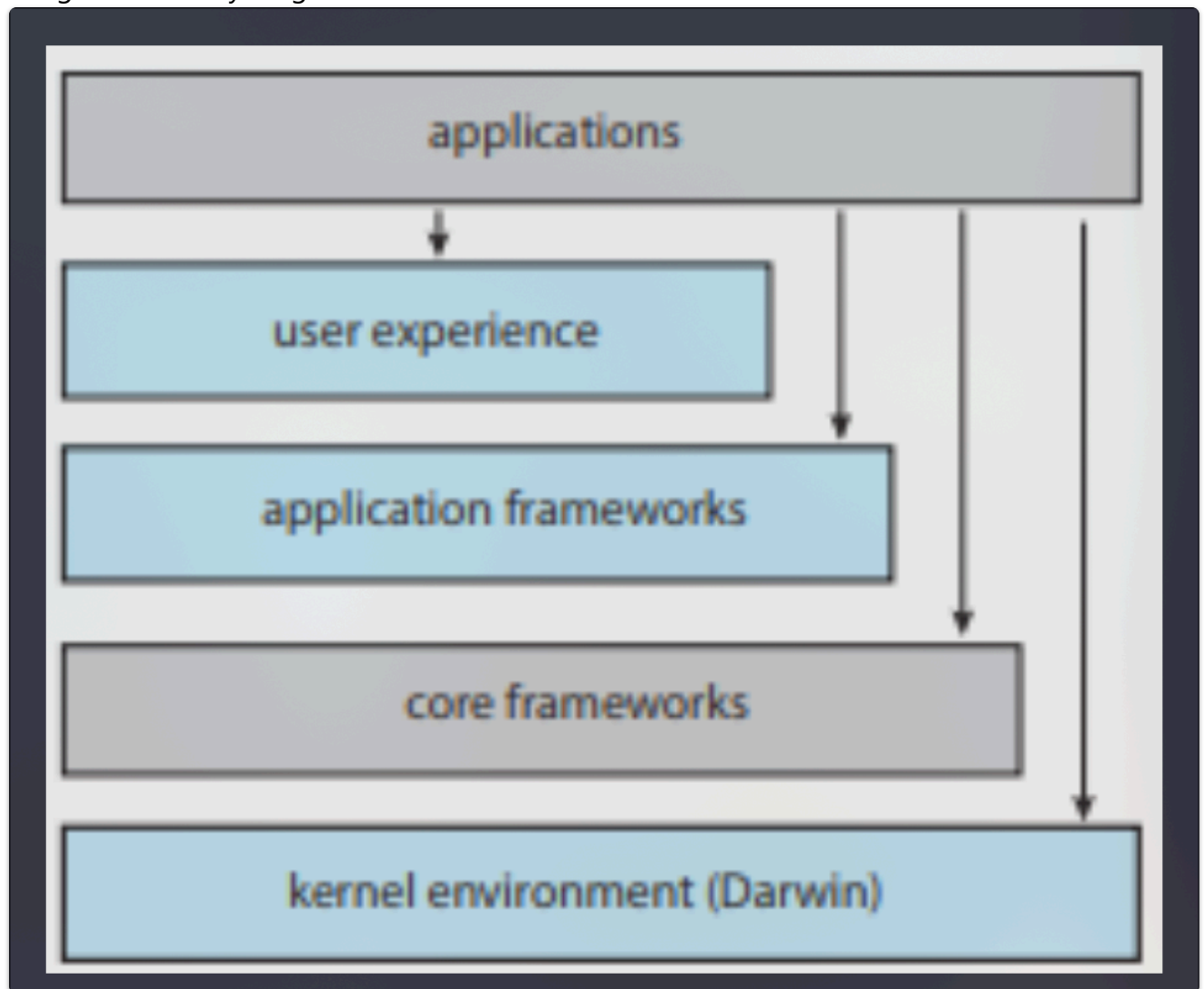
- If service fails, OS is left untouched
- Darwin, used by macOS and iOS

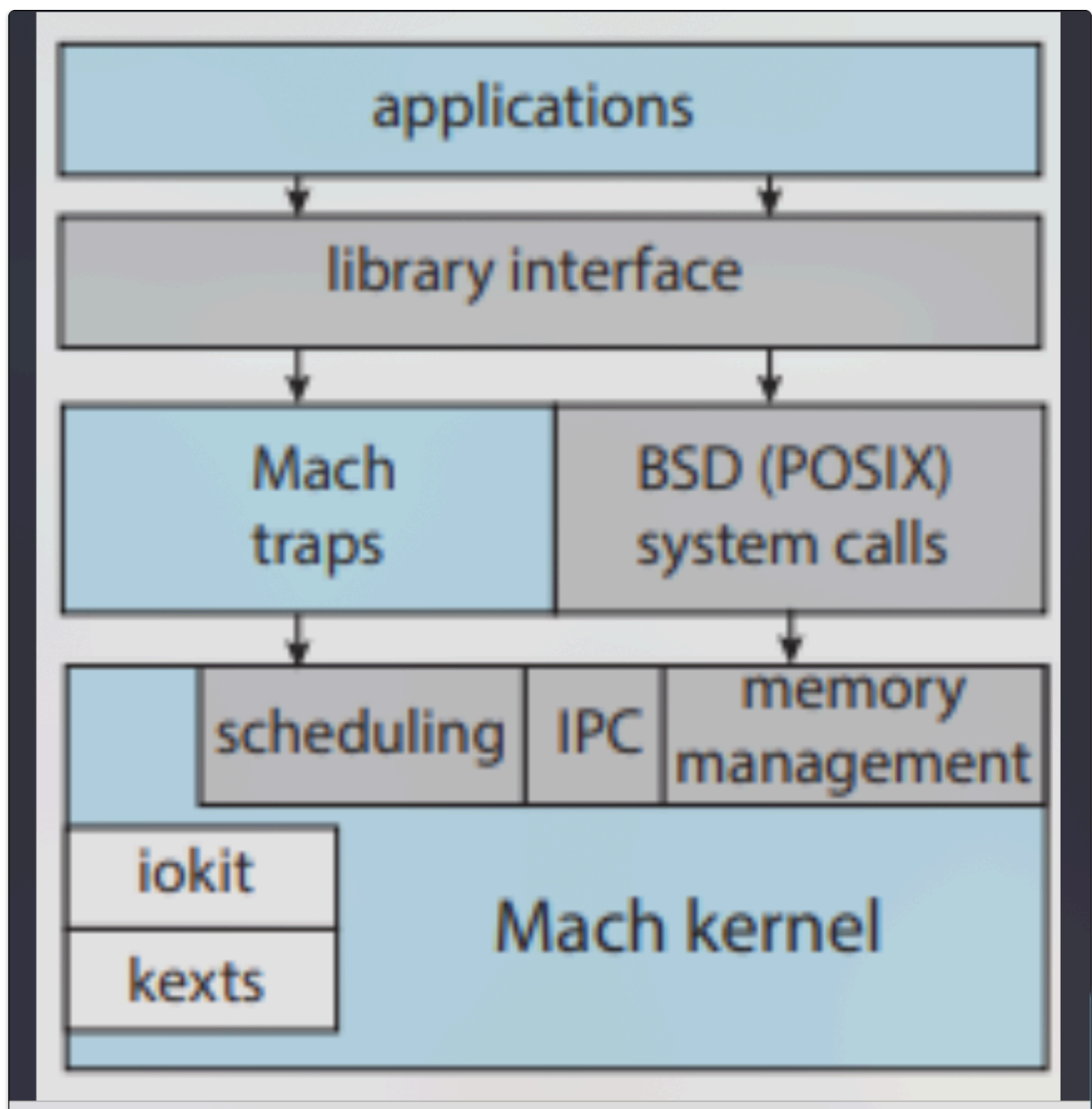
macOS and iOS Structure

- **User experience layer.** This layer defines the software interface that allows users to interact with the computing devices. macOS uses the Aqua user interface, which is designed for a mouse or trackpad, whereas iOS uses the Springboard user interface, which is designed for touch devices.
- **Application frameworks layer.** This layer includes the Cocoa and Cocoa Touch frameworks, which provide an API for the Objective-C and Swift programming languages. The primary difference between Cocoa and Cocoa Touch is that the former is used for developing macOS applications, and the latter by iOS to provide support for hardware features unique to mobile devices, such as touch screens.
- **Core frameworks.** This layer defines frameworks that support graphics and media including, Quicktime and OpenGL.
- **Kernel environment.** This environment, also known as Darwin, includes the Mach microkernel and the BSD UNIX kernel.

Image 7: macOS and iOS structure

Images below: Layering of macOS and iOS

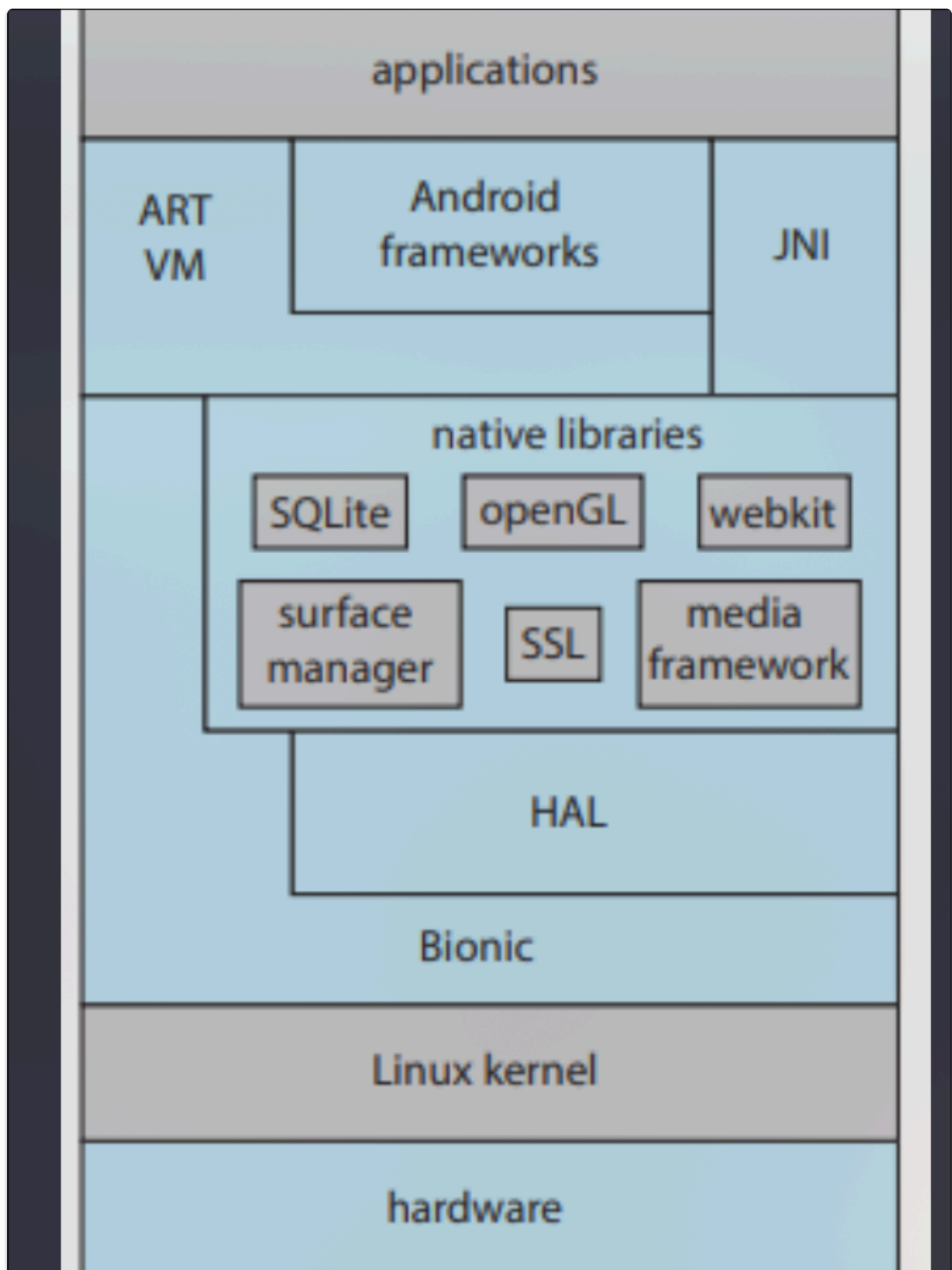




Android Structure

Android Structure

- Virtualized execution environment via Android Runtime ART (formerly Dalvik)
- Java programs are compiled to Java bytecode .class file and then translated to an executable .dex file.
- ART performs **ahead-of-time** compilation for performance considerations
- .dex file will be compiled to native machine code after installation on a device.
- Java Native Interface JNI is available such that developers can bypass the virtual machine, but not portable
- **Hardware Abstraction Layer (HAL)** abstracts hardware to the developers as Android may run on almost unlimited number of devices
- **Glibc** was replaced by **Bionic C** library for android
- Mobile optimized **Linux kernel** at the bottom of the stack



Virtual Machines

- Layered approach
- Operating system on top of another operating system

- May overhead but minimized because modern processors would have this Virtualization instruction set.

Intel® Virtualization Technology with Redirect Protection (VT-rp) [†]	Yes
Intel® Virtualization Technology (VT-x) [†] ⓘ	Yes
Intel® Virtualization Technology for Directed I/O (VT-d) [†] ⓘ	Yes
Intel® VT-x with Extended Page Tables (EPT) [†] ⓘ	Yes

Image 8: *Virtualization instruction set from Intel*

Virtualization instruction set allows us to run virtual environments as if they're native, removing that layering.